Template in C++

Teoria completa

Appunti di Ingegneria del Software

Basato sulle slide del corso (*Templates in C++*) e sulla voce Wikipedia *Template (C++)*

Function template, class template, specializzazione, template variadici, template metaprogramming, concepts

Anno 3

Contents

Chapter 1 Perche' servono i template — Page 4

1.1 Il problema: duplicazione di codice 4

1.2 Container class: le tre domande da porsi 4

Soluzioni pre-OO — 4 • Soluzioni OO — 5

Chapter 2 Cosa sono i template — Page 6

2.1 Definizione e scopo 6

Origini storiche e potere espressivo — 6

2.2 "I template non sono fully generic" 6

2.3 Quando viene generato codice 7

2.4 Istanziazione: il meccanismo generale 7

Chapter 3 Function template — Page 9

3.1 Definizione e motivazione 9

3.2 Perche' non basta l'overloading 9

3.3 Sintassi di dichiarazione 9

3.4 Sintassi di definizione 9

3.5 Istanziazione delle function template 10

3.6 Disambiguazione manuale 11

Chapter 4 Class template — Page 12

4.1 Definizione 12

4.2 La differenza fondamentale: nessuna deduzione 12

4.3 Specializzazioni come classi indipendenti 12

4.4 Istanziazione implicita ed esplicita 13

Chapter 5 Template specialization — Page 15

5.1 Cos'e' e perche' serve 15

Le due forme di specializzazione (da Wikipedia) — 16

Chapter 6 Oltre le slide: funzionalita' moderne (da Wikipedia) — Page 17

6.1 Variable template (C++14) 17

6.2 Parametri di template non-type 17
6.3 Template variadici (C++11) 17
6.4 Template alias (C++11) 18
6.5 Function template abbreviati e concepts (C++20) 18
6.6 Template metaprogramming 18
6.7 Una parentesi storica: gli "exported template" 19

Chapter 7 Genericita' in Java Page 20
7.1 Motivazione storica 20
7.2 Le quattro forme di genericita' (dalla JLS) 20
Diamond operator e varargs generici — 21 • Restrizione sui tipi primitivi — 21
7.3 Wildcard 21
7.4 Covarianza degli array vs invarianza dei generics 22
7.5 Type erasure: il meccanismo 22
Conseguenze dell'erasure — 22 • Perche' l'erasure: retrocompatibilita' — 23
7.6 Confronto esplicito con i template C++ (da Wikipedia) 23
7.7 Sviluppi futuri: Project Valhalla 23

Chapter 8 Genericita' in Python Page 24
8.1 Il punto di vista di Wikipedia: genericita' "gratuita" 24
8.2 Duck typing 24
8.3 Il collegamento teorico con i template: lo stesso "duck test", due momenti diversi 24
8.4 Structural typing 25

Chapter 9 Genericita' in C Page 26
9.1 "Function overloading, non true generic programming" 26
9.2 L'esempio GNU C: macro + `typeof` + statement expression 26
9.3 Il quadro completo delle tecniche C (slide + Wikipedia) 26

Chapter 10 Templates in Depth (dal libro di Eckel) Page 28
10.1 Parametri di template: le quattro forme 28
10.2 La keyword `typename` e i nomi dipendenti 29
10.3 Overloading e ordinamento parziale 29
10.4 Gli idiomi: traits, policies, CRTP 29
10.5 Metaprogramming ed expression templates 30
10.6 I due modelli di compilazione 30

Chapter 11 Sintesi trasversale Page 32
11.1 Tabella comparativa a quattro vie 33
11.2 Il filo che lega tutto 33

Chapter L'esercizio proposto dal corso Page 34

Chapter Riepilogo Page 36

Fonti 36

Chapter 1

Perche' servono i template

1.1 Il problema: duplicazione di codice

Molti programmi usano strutture dati comuni: stack, code, liste. Il problema di partenza, cosi' come lo pone il corso, e' molto concreto: un programma ha bisogno di una **coda di clienti**; poco dopo serve anche una **coda di messaggi**. La tentazione naturale e' prendere il codice della coda di clienti e, con un copia-incolla-trova-sostituisci, produrre una coda di messaggi. Quando il programma cresce ancora e serve una **coda di ordini**, la stessa operazione va ripetuta un'altra volta.

Fonte: slide TemplatesInCpp*, pag. 3.*

Note:

Il problema non e' scrivere la prima coda: e' che il codice viene *duplicato*. Se in seguito serve modificare l'implementazione della coda (correggere un bug, aggiungere un controllo), la modifica va replicata manualmente in ogni copia — ed e' facile perderne una. Ri-inventare codice sorgente identico e' l'opposto di cio' che l'orientamento agli oggetti promette: il **riuso**.

Definition 1.1.1: Type parameterization (genericita')

L'idea alternativa alla duplicazione: implementare la coda in modo che possa contenere **un tipo arbitrario**, parametrizzando il codice rispetto al tipo degli elementi. Questa tecnica e' comunemente chiamata **generics**; in C++ si chiama **template**.

Fonte: slide TemplatesInCpp*, pag. 4.*

1.2 Container class: le tre domande da porsi

Le slide inquadrano il problema in modo piu' generale, parlando di *container class* riusabili (liste, stack, code, insiemi, alberi, dizionari). Costruire container riusabili richiede di rispondere a tre domande:

1. E' possibile costruire abstraction di container **indipendenti dal tipo** dei loro elementi?
2. Questi container possono mantenere **un solo tipo** di valore (container *omogenei*) oppure **tipi diversi** contemporaneamente (container *eterogenei*)?
3. E' possibile dare agli utenti del container accesso agli elementi **senza rimuoverli** e **senza esporre** i dettagli implementativi del container?

Fonte: slide TemplatesInCpp*, pag. 6.*

1.2.1 Soluzioni pre-OO

In un linguaggio a **tipizzazione dinamica**, sia le abstraction omogenee sia quelle eterogenee si costruiscono con relativa facilita'. Nei linguaggi a **tipizzazione statica** pre-OO, invece:

- il tipo degli elementi del container e' *fisso*, quindi il riuso e' limitato;
- **variant record** o **union** aiutano fino a un certo punto a costruire container eterogenei, ma bisogna convivere con un **numero finito di alternative** e scrivere codice di test e cast "sporco" per estrarre i valori;
- nascondere i dettagli implementativi e' difficile: per iterare sul container serve introdurre una variabile che, con il suo tipo, rivela la struttura interna del container.

Fonte: slide TemplatesInCpp*, pag. 7.*

1.2.2 Soluzioni OO

Nei linguaggi OO a tipizzazione statica si possono costruire container sia omogenei sia eterogenei, a patto che:

- il linguaggio supporti la **substitutability**, con tutti i valori istanze di sottoclassi di una comune superclasse `general-element`;
- gli oggetti conoscano il proprio **tipo dinamico** — test e cast espliciti restano necessari per estrarre i valori, ma possono essere nascosti tramite sotto-classing.

Si possono introdurre **iteratori espliciti** per scorrere gli elementi del container senza esporne la struttura implementativa interna.

> **Proposition 1.2.1** Per liste omogenee, i template SONO "la" soluzione
>
> Questa e' l'affermazione centrale delle slide: per costruire container **omogenei** in un linguaggio a tipizzazione statica, i template (o i generics) sono la soluzione di riferimento.

Fonte: slide TemplatesInCpp*, pag. 8.*

Chapter 2

Cosa sono i template

2.1 Definizione e scopo

Definition 2.1.1: Template

I template forniscono un modo di **parametrizzare una classe o una funzione con un tipo**. Sono stati introdotti nei linguaggi a tipizzazione statica per permettere la creazione e la manipolazione di strutture dati generiche, come le container class riusabili.

Fonte: slide TemplatesInCpp*, pag. 10.*

E' il **compilatore** che genera il codice corretto per ogni **istanziazione** del template: il corpo effettivamente eseguito non e' generico, ma *molto specifico* per quel particolare tipo.
I template sono usati estesamente nella **Standard Template Library** (STL). Un singolo template da' origine a una **famiglia di funzioni o classi**: tutte compilate a partire dallo stesso codice sorgente, ma che operano su tipi diversi.
Fonte: slide TemplatesInCpp*, pag. 11.*

Note:

Secondo Wikipedia, dal punto di vista concettuale un template equivale, prima della compilazione, a *copiare e incollare* il blocco di codice templatizzato nel punto in cui viene usato, sostituendo poi il parametro di template con il tipo attuale. Questa e' esattamente l'immagine mentale corretta per capire perche' il codice generato "non e' generico ma molto specifico".

Fonte: Wikipedia, Template (C++).

2.1.1 Origini storiche e potere espressivo

Le principali influenze storiche sui template C++ sono i **moduli parametrizzati** del linguaggio CLU e i **generics** di Ada, che precedono i template C++. I template C++ sono inoltre dimostrati essere **Turing completi**: il sistema dei template, da solo, e' potente abbastanza da poter esprimere qualunque computazione calcolabile — proprio questa proprieta' rende possibile il *template metaprogramming* (capitolo 6.6).
Fonte: Wikipedia, Template (C++).

2.2 "I template non sono fully generic"

Wrong Concept 2.1: I template sono un mccanismo completamente generico

Le slide affermano esplicitamente il contrario: **i template non sono fully generic**. L'obiettivo e' costruire selettivamente una classe assemblando solo le porzioni di codice necessarie, mantenendo **solo il necessario**. Per essere efficiente, questo meccanismo deve essere eseguito **a compile-time**, proprio per poter sopprimere la generazione di codice inutile — e infatti **non c'e' type checking completo** alla

definizione del template.

Fonte: slide TemplatesInCpp, *pag. 12.*

2.3 Quando viene generato codice

Proposition 2.3.1 Nessun codice alla definizione

Nessun codice viene generato quando il compilatore incontra la *definizione* di un template. Il compilatore esegue in quella fase ogni controllo possibile, ma **rimanda** il type checking completo e la compilazione effettiva alla fase di **istanziazione**.

Di conseguenza: le dichiarazioni e le definizioni dei metodi membro di una class template devono stare tutte nello **stesso header file**.
Fonte: slide TemplatesInCpp, *pag. 13.*

Example 2.3.1 (La tecnicalita' declaration/definition/instantiation)

Il motivo concreto per cui serve questa regola emerge da un esempio a tre file:

```
// B.H  -- Declaration
template <class t>
class b {
public:
    b();
    ~b();
};

// B.CPP -- Definition
#include "B.H"
template <class t> b<t>::b()  { }
template <class t> b<t>::~b() { }

// MAIN.CPP -- Instantiation
#include "B.H"
void main() {
    b<int>   bi;
    b<float> bf;
}
```

Quando si compila `B.CPP`, il compilatore ha a disposizione sia le dichiarazioni sia le definizioni: a questo punto **non** deve generare alcuna definizione per la classe template, perche' non c'e' ancora nessuna istanziazione. Quando invece si compila `MAIN.CPP` ci sono due istanziazioni (`b<int>` e `b<float>`), ma in quel file il compilatore ha solo le **dichiarazioni**, non le definizioni: se dichiarazione e definizione stessero in file diversi (come qui), il linker non troverebbe il codice generato. Ecco perche' per i template si tengono dichiarazione e definizione nello **stesso header**.

Fonte: slide TemplatesInCpp, *pag. 14.*

2.4 Istanziazione: il meccanismo generale

Definition 2.4.1: Punto di istanziazione (instantiation point)

Il punto nel codice dove un template viene effettivamente **usato**. Li' il compilatore conosce i tipi attuali corrispondenti ai parametri del template, e **genera il codice** per quei tipi specifici. Questo processo si chiama **template instantiation**; il codice generato e' una *generated instance* del template.

Una classe generata da una class template si chiama **generated class**; una funzione generata da una function template si chiama **generated function**; un dato statico generato da un membro statico di template si chiama **generated static data member**.

Proposition 2.4.1 Generazione una tantum

Una generated instance viene generata **una sola volta**. Una volta generata, viene **riusata** a ogni successiva istanziazione con gli stessi parametri.

L'istanziazione di un template puo' avvenire in due modi:

- **implicitamente**: basta usare il template con dei tipi concreti, es. `Z<int> zi;`
- **esplicitamente**: con la sintassi `template class Z<int>;`

Fonte: slide TemplatesInCpp*, pag. 15.*

Example 2.4.1 (Istanziazione implicita)

```
template <class T> class Z {
public:
    Z()  {}
    ~Z() {}
    void f() {}
    void g() {}
};

int main() {
    Z<int>   zi;    // instantiation generates class Z<int>
    zi.f();         // and generates function Z<int>::f()
    Z<float> zf;    // instantiation generates class Z<float>
    zf.g();         // and generates function Z<float>::g()
    return 0;
}
```

Da notare: `Z<int>::f()` viene generata quando viene *chiamata*, non semplicemente perche' esiste la classe `Z<int>`; `Z<float>::g()` viene generata a sua volta solo quando viene chiamata — e non `Z<float>::f()` o `Z<int>::g()`, che nel programma non vengono mai usate.

Fonte: slide TemplatesInCpp*, pag. 16.*

Example 2.4.2 (Un puntatore non forza l'istanziazione)

```
template <class T> class Z {
public:
    Z()  {} ; ~Z() {} ; void f(){} ; void g(){} ;
};
int main() {
    Z<int>*   p_zi;   // instantiation of class Z<int> NOT required
    Z<float>* p_zf;   // instantiation of class Z<float> NOT required
    return 0;
}
```

Questa volta il compilatore **non genera alcuna definizione**: non ce n'e' bisogno. E' analogo a dichiarare un puntatore a una classe o struct **non ancora definita** — il compilatore deve solo sapere che `Z<int>` e' un tipo, non serve conoscerne il contenuto per dichiararne un puntatore.

Fonte: slide TemplatesInCpp*, pag. 24.*

Chapter 3

Function template

3.1 Definizione e motivazione

Definition 3.1.1: Function template

Permette di definire una cosiddetta **generic function**, applicabile ad argomenti i cui tipi possono essere specificati piu' tardi, durante la *chiamata* alla funzione. In questo modo si informa il compilatore di **non** eseguire il controllo completo di consistenza dei tipi della definizione della funzione finche' non si incontra una chiamata: e' un modo di **rilassare il type checking**.

Fonte: slide TemplatesInCpp*, pag. 18.*

3.2 Perche' non basta l'overloading

Con l'**overloading**, il comportamento di una funzione deve essere completamente specificato, incluso il tipo dei suoi argomenti: il comportamento deve essere codificato **in anticipo** per ogni tipo di parametro. Se, nel momento in cui si *scrive* la funzione, il tipo degli argomenti attuali non e' ancora noto, l'overloading semplicemente **non si puo' usare**.
Fonte: slide TemplatesInCpp*, pag. 19.*

3.3 Sintassi di dichiarazione

Una *dichiarazione* di function template e' molto simile a quella di una funzione normale, a parte la keyword `template` seguita dalla lista dei parametri di template (tra `<>`), che deve apparire prima della dichiarazione della funzione:

```
template <typename T1, typename T2>
    void f(T1 t, T2* p);
```

La keyword `typename` puo' essere sostituita, in questo contesto, dalla keyword `class` — ma questo uso e' **deprecato**; si trova spesso in codice vecchio, pre-standard.
Qui si dichiara la function template `f`, con due argomenti `t` e `p` di tipo `T1` e "puntatore a `T2`" rispettivamente. `T1` e `T2` sono i **parametri di template**: agiscono come placeholder per nomi di tipo attuali, che verranno specificati alla chiamata della function template.
Fonte: slide TemplatesInCpp*, pag. 20.*

3.4 Sintassi di definizione

La *definizione* di una function template segue la stessa sintassi della dichiarazione per quanto riguarda il prototipo; il **corpo** della definizione e' invece sintatticamente identico al corpo di una funzione normale. All'interno del corpo, i parametri di template sono trattati **esattamente come nomi di tipo**, dal punto di vista sintattico.

Il compilatore non puo' eseguire, *in questa fase*, tutti i controlli che eseguirebbe su nomi di tipo reali, perche' i parametri di template non specificano ancora, in questa fase, un tipo vero e completamente definito.
Fonte: slide TemplatesInCpp*, pag. 21.*

Example 3.4.1 (Il classico esempio `r = 2 * t`)

```
template <typename T1, typename T2> void f(T1 t, T2& r) {
    r = 2 * t;
}
```

Qui si dichiara una function template che esegue una semplice operazione aritmetica. Non sapendo cosa siano realmente `T1` e `T2`, il compilatore **non puo' controllare**, mentre analizza questo codice, se l'espressione `r = 2 * t` sia legale. Sarebbe legale se `T1` fosse `int` o `float` e `T2` fosse `float`; sarebbe invece **illegale** se `T1` fosse un tipo array arbitrario (perche' un array non si puo' moltiplicare per 2, come invece si tenta di fare con `t`). E' quindi chiaro che alcuni controlli devono essere rimandati fino a quando il template non viene **istanziato**.

Fonte: slide TemplatesInCpp*, pag. 22.*

3.5 Istanziazione delle function template

Le function template non vengono effettivamente compilate finche' non sono **esplicitamente istanziate**, cioe' usate per eseguire una chiamata. Per chiamare una function template, la sintassi formale e', quasi sempre, identica a quella delle chiamate a funzioni normali.
Il compilatore riesce *di solito* a **dedurre automaticamente** i tipi attuali (i "valori" dei parametri di template), usando algoritmi di pattern matching piuttosto complessi, esaminando i tipi degli argomenti effettivi fornito nella chiamata.
Fonte: slide TemplatesInCpp*, pag. 23.*

Example 3.5.1 (Deduzione corretta ed errore rilevato solo all'istanziazione)

```
template <typename T1, typename T2> void f(T1 t, T2& r) {
    r = 2 * t;
}
class A {};   // empty class; for illustration purposes

int main() {
    int x = 1;
    int y = 2;
    f(y, x);     // OK
    A anA;
    f(anA, x);   // WRONG
}
```

`f(y, x)` **compila**: il compilatore deduce facilmente che `T1` e' `int` e `T2` e' "reference to int", perche' sia `y` sia `x` sono `int`. `f(anA, x)` invece **non compila** (e il compilatore emette un errore proprio in questo punto): il compilatore deduce senza problemi che `T1` e' `A` e `T2` e' "reference to int" (perche' `anA` e' una `A` e `x` e' un `int`), ma la funzione tenterebbe poi di moltiplicare per 2 una copia di `anA`, operazione impossibile perche' la classe `A` non sovraccarica `operator*` in modo adatto. E' esattamente l'errore che, per costruzione, non poteva essere rilevato alla definizione del template (sezione precedente): emerge solo ora, all'istanziazione.

Fonte: slide TemplatesInCpp*, pag. 25.*

Example 3.5.2 (`max`: deduzione del tipo di ritorno)

```
template <class T> T max(T a, T b) {
    return a > b ? a : b;
}
```

```
void main() {
    int I;
    I = max(10, 15);
    cout << "max(10, 15) = " << I << endl;
    char c;
    c = max('k', 's');
    cout << " max('k', 's') = " << c << endl;
}
```

Output: `max(10, 15) = 15` (`T` dedotto `int`); `max('k', 's') = s` (`T` dedotto `char`, confronto lessicografico sui codici carattere).

Fonte: slide TemplatesInCpp*, pag. 26.*

3.6 Disambiguazione manuale

Ci sono casi in cui il compilatore **non riesce** a dedurre il tipo attuale del parametro di template dalla chiamata, perche' gli argomenti attuali della funzione non portano abbastanza informazione di tipo. In questi casi il compilatore emette un errore, segnalando la chiamata come **ambigua** — anche se la chiamata sarebbe eseguibile, conoscendo i tipi da assegnare ai parametri di template.
E' quindi necessario **disambiguare la chiamata manualmente**, specificando i "valori" dei parametri di template in una lista (tra <>) subito dopo il nome della funzione e prima della lista degli argomenti:

```
template_func_name<parameter_list>(arguments_list)
```

Fonte: slide TemplatesInCpp*, pag. 27.*

Chapter 4

Class template

4.1 Definizione

Definition 4.1.1: Class template

Una classe **parametrizzata rispetto a uno o piu' tipi**. La sintassi per definire una class template e' quasi identica a quella delle function template.

Fonte: slide TemplatesInCpp, *pag. 29.*

4.2 La differenza fondamentale: nessuna deduzione

Proposition 4.2.1 Il compilatore non deduce i parametri di una class template

La differenza principale fra function template e class template: per le **classi** il compilatore **non tenta** di dedurre i parametri di template. La deduzione sarebbe comunque **impossibile**: non c'e' nessuna informazione aggiuntiva su cui basarsi per dedurli (nel caso delle function template c'e' la lista degli argomenti attuali della chiamata, su cui la deduzione si appoggia).

Ogni volta che una class template viene istanziata, i "valori" (cioe' i tipi attuali) dei parametri di template devono essere **specificati**.
Fonte: slide TemplatesInCpp, *pag. 30.*

4.3 Specializzazioni come classi indipendenti

Definition 4.3.1: Specializzazione (di class template)

Ogni volta che il compilatore istanzia una class template con parametri attuali, crea una nuova **specializzazione** — se una specializzazione con quei parametri non esiste gia' (altrimenti viene riusata).

Proposition 4.3.1 Due specializzazioni sono classi scorrelate

Due specializzazioni diverse della stessa class template sono classi **regolari**, e sono **completamente scorrelate** anche se generate dallo stesso template. Se una class template e' dichiarata come `A<typename T>`, allora `A<int>`, `A<double>` e `A<B>` (con `B` un'altra classe) sono classi completamente indipendenti fra loro.

Conseguenza pratica: **non** si puo' assegnare una variabile di tipo `A<int>` a una variabile di tipo `A<double>`, a meno di fornire un **operatore di conversione esplicito**.
Fonte: slide TemplatesInCpp, *pag. 31.*

4.4 Istanziazione implicita ed esplicita

implicita	esplicita

`Z<int> zi;`
`// implicit instantiation`
`// generates class Z<int>`
	`template class Z<int>;`
	`// explicit instantiation`
	`// of class Z<int>`

Fonte: slide TemplatesInCpp*, pag. 32.*

Example 4.4.1 (Class template completa: `A<T>`)

```
template <typename T> class A {
    T x, y;
public:
    A() { x = 0; y = 0; }
    T getSum() { return x + y; }
    void setXY(T a, T b) { x = a; y = b; }
};

int main() {
    A<int>    iA;
    A<double> dA;
    int y    = iA.getSum();
    double w = dA.getSum();
    iA.setXY(1,2);
    dA.setXY(0.0,0.2);
}
```

I parametri attuali di template devono essere specificati esplicitamente: `A<int>` e `A<double>` sono tipi completamente diversi, che in generale non si possono mescolare tra loro.

Fonte: slide TemplatesInCpp*, pag. 33.*

Example 4.4.2 (Il codice che il compilatore genera davvero)

Per la classe template dell'esempio precedente, il compilatore genera — a tutti gli effetti — due classi regolari indipendenti:

```
class A_int {
    int x, y;
public:
    A_int() { x = 0; y = 0; }
    int getSum() { return x + y; }
    void setXY(int a, int b) { x = a; y = b; }
};
class A_double {
    double x, y;
public:
    A_double() { x = 0; y = 0; }
    double getSum() { return x + y; }
    void setXY(double a, double b) { x = a; y = b; }
};
int main() {
    A_int    iA;
    A_double dA;
    int    y = iA.getSum();
    double w = dA.getSum();
    iA.setXY(1,2);
    dA.setXY(0.0,0.2);
}
```

(I nomi `A_int`/`A_double` sono qui solo illustrativi: nel binario reale i simboli generati sono *mangled*, ma il

punto delle slide e' proprio questo — il corpo eseguito **non e' generico**, e' codice specifico duplicato per ciascun tipo istanziato.)

Fonte: slide TemplatesInCpp*, pag. 34.*

Chapter 5

Template specialization

5.1 Cos'e' e perche' serve

In alcuni casi e' possibile **sovrascrivere** il codice generato dal template, fornendo una definizione speciale per un tipo specifico. Questo si chiama **template specialization**.
Fonte: slide TemplatesInCpp, *pag. 35.*

Wrong Concept 5.1: Il comportamento generico del template va sempre bene per ogni tipo

Le slide costruiscono apposta un controesempio per mostrare che non e' cosi': un algoritmo scritto in modo completamente generico puo' comportarsi in modo tecnicamente corretto ma **semanticamente sbagliato** per un tipo particolare.

Example 5.1.1 (Il tranello `max("Aladdin", "Jasmine")`)

```
// max restituisce il massimo dei due elementi di tipo T, dove T e'
// una classe o un tipo per cui operator> e' definito
template <class T> T max(T a, T b) {
    return a > b ? a : b;
}
int main() {
    cout << "max(10, 15) = "     << max(10, 15)     << endl;
    cout << "max('k', 's') = "   << max('k', 's')   << endl;
    cout << "max(10.1, 15.2) = " << max(10.1, 15.2) << endl;
    cout << "max(\"Aladdin\", \"Jasmine\") = "
         << max("Aladdin", "Jasmine") << endl;
}
```

I primi tre risultati sono quelli attesi: `15`, `s`, `15.2`. Il quarto **non** e' quello atteso (non stampa `"Jasmine"`, che viene dopo alfabeticamente): la chiamata `max("Aladdin","Jasmine")` fa istanziare il template con `T = char*`, cioe' `max(char*, char*)`, che confronta gli **indirizzi di memoria** dei due puntatori, non il **contenuto** delle stringhe che puntano.

Fonte: slide TemplatesInCpp, *pag. 36-37.*

Example 5.1.2 (La correzione tramite specializzazione)

```
// Specialization of max for char*
template <> char* max(char* a, char* b) {
    return strcmp(a, b) > 0 ? a : b;
}
```

La specializzazione **sostituisce** la versione generica ogni volta che `max` viene chiamata con due `char*`, usando `strcmp` per un vero confronto del contenuto delle stringhe invece del confronto (privo di significato

logico) tra i loro indirizzi.
Fonte: Wikipedia, Template (C++).
Secondo Wikipedia, questo stesso esempio, riportato nella versione moderna del linguaggio, usa `std::strcmp` per fornire un comportamento *ben definito* nel confronto di C-string — il confronto diretto tra puntatori sarebbe altrimenti privo di significato utile. In C++ moderno il problema semplicemente **non si presenta** usando `std::string` al posto di `char*`, perche' `std::string` sovraccarica correttamente `operator>` confrontando il contenuto.

5.1.1 Le due forme di specializzazione (da Wikipedia)

Definition 5.1.1: Full specialization vs partial specialization

Full specialization: tutti i parametri di template vengono specializzati (l'esempio di `max(char*, char*)` sopra e' una full specialization). **Partial specialization**: solo un *sottoinsieme* dei parametri viene specializzato. La specializzazione parziale e' disponibile **solo per le class template**: le function template non possono essere specializzate parzialmente (si usa l'overloading al loro posto).

Fonte: Wikipedia, Template (C++).

Chapter 6

Oltre le slide: funzionalita' moderne (da Wikipedia)

Il corso presenta i template nella loro forma "classica". La voce Wikipedia *Template (C++)* descrive anche le evoluzioni introdotte dagli standard successivi (C++11, 14, 17, 20), utili per capire dove i template sono andati dopo le basi viste finora. Tutto questo capitolo ha come unica fonte Wikipedia.
Fonte: Wikipedia, Template (C++).

6.1 Variable template (C++14)

Da C++14 anche le **variabili** possono essere templatizzate:

```
template <typename T>
constexpr T PI = T{3.14159265358979323846264338 3L};
```

`PI<float>` e `PI<double>` sono due costanti distinte, ciascuna del tipo richiesto, generate a compile time.

6.2 Parametri di template non-type

Un template puo' essere parametrizzato anche su un **valore**, non solo su un tipo:

```
template <int K>
class MyClass;
```

L'esempio reale piu' importante e' `std::array` della libreria standard:

```
template <class T, size_t N>
struct Array;

Array<char, 6> myArray;   // array di 6 caratteri
```

`std::array` e' parametrizzato sia su un tipo (`T`) sia su un valore intero (`N`, la dimensione): a differenza di un array C classico, la dimensione fa parte del **tipo** dell'oggetto.

6.3 Template variadici (C++11)

I template variadici accettano un **numero variabile** di parametri di template, in modo analogo alle funzioni variadiche in stile `printf`.

Definition 6.3.1: Vantaggio principale

In C++ sono supportati solo i parametri variadici "in stile C" (...); l'unico modo per ottenere funzioni variadiche **type-safe** (come in Java) e' proprio tramite i template variadici.

```
template <typename... Args>
void log(const format_string<Args...>& fmt, Args&&... args) {
    std::println(logFile, fmt, args...);
}
```

6.4 Template alias (C++11)

Un template alias funziona come un **typedef parametrizzato**, utile per dare un nome comodo a un'istanziazione parziale di un template piu' complesso:

```
template <typename K, typename V>
using TreeMap = std::map<K, V>;

template <typename K, typename V>
using HashMap = std::unordered_map<K, V>;

// si puo' anche fissare solo alcuni parametri:
template <typename K>
using StringHashMap = HashMap<K, std::string>;

StringHashMap<int> myMap;   // == std::unordered_map<int, std::string>
```

6.5 Function template abbreviati e concepts (C++20)

Da C++20, usare `auto` (eventualmente vincolato da un *concept*) come tipo di un parametro di funzione dichiara implicitamente una function template abbreviata:

```
void f1(auto x);                // == template <typename T> void f1(T x);
void f2(Concept1 auto x);       // == template <Concept1 T> void f2(T x);
void f3(Concept2 auto... xs);   // variadico, vincolato da Concept2
```

Definition 6.5.1: Concept

Un **predicato booleano valutato a compile time**, usato per vincolare i parametri di template. Sono l'equivalente C++ dei wildcard di Java o delle clausole `where` di Rust.

```
template <std::derived_from<Player> T>
void processListOfPlayers(const std::vector<T>& players) { /* ... */ }
```

Qui il compilatore **rifiuta** qualunque `T` che non derivi da `Player` — un vincolo espresso direttamente nella dichiarazione del template, invece che scoperto tardi tramite un errore di sostituzione oscuro (come succedeva, storicamente, con SFINAE).

6.6 Template metaprogramming

Definition 6.6.1: Template metaprogramming

Usare il meccanismo dei template per eseguire **computazioni a compile time**, sfruttando il fatto che i template C++ sono Turing completi (capitolo 2).

Example 6.6.1 (Fattoriale a compile time, stile classico (specializzazione))

```
// caso induttivo
template <unsigned int N>
struct Factorial {
    static constexpr unsigned int value = N * Factorial<N - 1>::value;
};
// caso base, tramite full specialization
template <>
struct Factorial<0> {
    static constexpr unsigned int value = 1;
};
// uso: Factorial<6>::value calcola 6! a compile time
```

Questo e' lo stile "storico" di metaprogramming: si usa la **specializzazione** (capitolo 5) come se fosse un caso base di una ricorsione, esattamente come si farebbe a runtime con una funzione ricorsiva.

Example 6.6.2 (Lo stesso fattoriale, stile moderno (`constexpr` + `if constexpr`))

```
template <unsigned int N>
constexpr unsigned int factorial() noexcept {
    if constexpr (N <= 1) {
        return 1;
    } else {
        return N * factorial<N - 1>();
    }
}
```

Con `constexpr` e `if constexpr` (C++17) si ottiene lo stesso risultato con una sintassi molto piu' simile a una normale funzione ricorsiva, senza dover simulare l'`if` tramite la specializzazione. Per questo motivo oggi il template metaprogramming "puro" e' usato soprattutto per fare operazioni **sui tipi** (non sui valori), dove `constexpr` non basta.

6.7 Una parentesi storica: gli "exported template"

C++03 introdusse la keyword `export` per permettere di dichiarare un template in un header senza doverne fornire la definizione in ogni unita' di traduzione che lo usa (il vincolo visto nel capitolo 2). La feature venne **rimossa in C++11**, perche' quasi nessun compilatore la implementava davvero (solo Comeau C/C++), era costosa/complessa da implementare, dava scarso beneficio pratico, e limitava l'evoluzione futura del linguaggio. Con C++20 e i **moduli**, la keyword `export` e' stata reintrodotta, ma con un significato diverso (esportare simboli da un modulo): il beneficio di velocita' di compilazione che gli exported template avrebbero dovuto offrire e' oggi fornito dai moduli stessi, rendendo la vecchia feature "essenzialmente obsoleta".
Fonte: Wikipedia, Template (C++).

Chapter 7

Genericita' in Java

Il corso non ha slide dedicate ai generics Java: questo intero capitolo si appoggia alla voce Wikipedia *Generics in Java*.
Fonte: Wikipedia, Generics in Java.

7.1 Motivazione storica

I generics sono stati introdotti in Java nel **2004**, con J2SE 5.0. Nel 1998 Gilad Bracha, Martin Odersky, David Stoutamire e Philip Wadler avevano creato *Generic Java*, un'estensione del linguaggio con supporto per tipi generici; quel lavoro e' stato poi incorporato in Java, con l'aggiunta del supporto ai wildcard. La motivazione primaria era rendere **rilevabili a compile time** disallineamenti di tipo che prima emergevano solo a runtime.

Example 7.1.1 (Il problema che i generics risolvono)

```
List v = new ArrayList();
v.add("test");
Integer i = (Integer) v.get(0);   // ClassCastException a runtime
```

Il programmatore inserisce una `String` ma tenta un cast a `Integer`: un errore logico che, senza generics, non viene rilevato a compile time.

```
List<String> v = new ArrayList<String>();
v.add("test");
Integer i = (Integer) v.get(0);   // ERRORE rilevato a compile time
```

Il parametro di tipo tra parentesi angolari dichiara che la lista contiene *esclusivamente* `String`: il compilatore puo' ora verificare la correttezza dei tipi.

7.2 Le quattro forme di genericita' (dalla JLS)

Secondo la Java Language Specification, la genericita' coinvolge quattro elementi:

- **Generic class**: una classe che dichiara una o piu' variabili di tipo, che agiscono da parametri;
- **Generic interface**: analogo per le interfacce;
- **Generic method**: un metodo che dichiara variabili di tipo come parametri di tipo formali, con sintassi analoga a quella di classi/interfacce generiche;
- **Generic constructor**: un costruttore che dichiara indipendentemente le proprie variabili di tipo, permettendo un comportamento generico anche se la classe che lo contiene non e' generica.

Example 7.2.1 (Una classe generica `Entry<K,V>`)

```
public class Entry<K, V> {
    private K key;
    private V value;
    public Entry(K key, V value) { this.key = key; this.value = value; }
    public K getKey()   { return key; }
    public V getValue() { return value; }
    public String toString() { return String.format("(%s, %s)", key, value); }
}

Entry<String, String>  grade = new Entry<>("Mike", "A");
Entry<String, Integer> mark  = new Entry<>("Mike", 100);
Entry<Integer, Boolean> prime = new Entry<>(13, true);
```

E un metodo generico:

```
public static <T> Entry<T, T> twice(T value) {
    return new Entry<T, T>(value, value);
}

Entry<String, String> pair = Entry.twice("Hello");          // dedotto
Entry<String, String> pair2 = Entry.<String>twice("Hello");  // esplicito
```

7.2.1 Diamond operator e varargs generici

Da Java SE 7, l'**operatore diamond** (`<>`) permette di omettere i parametri di tipo ridondanti quando sono deducibili dal contesto (vedi `new Entry<>(...)` sopra).

Wrong Concept 7.1: Il diamond operator Java funziona come `Box<>` in C++

No: in C++ una sintassi simile a `Box<>` si riferisce a **default template argument** (es. `Box<int>` con un default), non a inferenza di tipo. Il diamond operator Java e' invece puro **type inference** al punto di chiamata. Anche C# e Kotlin non hanno affatto il diamond operator: usano sintassi alternativa per evitare la ripetizione ridondante dei tipi.

I metodi generici supportano anche i **varargs**:

```
public <T> T[] toArray(T... elements) { return elements; }
Integer[] array = toArray(1, 2, 3, 4, 5, 6);
```

7.2.2 Restrizione sui tipi primitivi

I tipi primitivi sono **vietati** come parametri di tipo: serve la classe wrapper. `Entry<int, int>` non compila; serve `Entry<Integer, Integer>`. Gli *array* di primitivi, essendo oggetti essi stessi, sono invece ammessi: `List<int[]>` e' legale.

7.3 Wildcard

I wildcard (`?`) sono argomenti di tipo per tipi parametrizzati, opzionalmente vincolati.

Example 7.3.1 (Le tre forme di wildcard)

```
// unbounded: puo' leggere solo Object, scrivere solo null
Collection<?> c = new ArrayList<String>();
c.add(new Object());   // ERRORE a compile time
c.add(null);           // permesso
```

```java
// upper-bounded (extends): "produttore", si legge
List<? extends Number> nums = new ArrayList<Float>();
nums.add(3.14);          // ERRORE
nums.add(null);           // permesso

// lower-bounded (super): "consumatore", si scrive
List<? super Number> nums2 = new ArrayList<Object>();
nums2.add(42);            // OK: 42 e' un Number
```

Definition 7.3.1: PECS (Producer Extends, Consumer Super)

Mnemonico di Joshua Bloch (*Effective Java*): usa `? extends` quando il parametrizzato **produce** valori (li leggi); usa `? super` quando **consuma** valori (li scrivi). Corrisponde esattamente alle nozioni di covarianza e controvarianza.

7.4 Covarianza degli array vs invarianza dei generics

Example 7.4.1 (Perche' i generics sono invarianti (a differenza degli array))

```java
Object[] objs = new Long[0];
objs[0] = "string";     // compila; ArrayStoreException A RUNTIME

Collection<Object> objs2 = new ArrayList<Long>();   // ERRORE a compile time
```

Gli array in Java sono **covarianti** (`Long[]` e' assegnabile a `Object[]`), ma questo costringe a un controllo a runtime che puo' fallire con `ArrayStoreException`. I generics sono invece deliberatamente **invarianti**: `ArrayList<Long>` non e' un `Collection<Object>`, e l'errore viene spostato interamente a compile time, prima ancora di eseguire il programma.

7.5 Type erasure: il meccanismo

Definition 7.5.1: Type erasure

L'informazione di tipo generico viene **rimossa durante la compilazione**. `List<Integer>` diventa il semplice `List` (non generico) nel bytecode compilato; la correttezza e' garantita dai controlli fatti a compile time, non da alcuna verifica residua a runtime.

```java
List<Integer> li = new ArrayList<>();
List<Float>   lf = new ArrayList<>();
if (li.getClass() == lf.getClass()) {
    System.out.println("Equal");   // stampa "Equal": a runtime sono la stessa classe
}
```

7.5.1 Conseguenze dell'erasure

- **Nessuna classe eccezione generica**: una classe generica non puo' estendere `Throwable`. Per erasure, un `catch (GenericException<Integer> e)` e un `catch (GenericException<String> e)` sarebbero indistinguibili — il secondo sarebbe codice **irraggiungibile**, quindi il linguaggio vieta la cosa a monte;
- **Nessuna creazione diretta di istanze**: `new T()` non compila, perche' a runtime `T` non esiste piu';
- **Restrizioni sui membri static**: esiste **una sola** copia a runtime di una classe generica; tutte le istanze condividono le variabili static indipendentemente dal parametro di tipo usato, quindi un parametro di tipo

non puo' apparire in una dichiarazione static;

- nei `throws`, un parametro di tipo e' ammesso solo se **vincolato** da `Throwable`:

```
public <T extends Throwable> void throwMeConditional(boolean c, T exception) throws T {
    if (c) throw exception;
}
```

Definition 7.5.2: Reificazione

Gli **array sono reificati**: portano l'informazione di tipo a runtime (ogni tipo `T` ha un corrispondente `T[]`). I **generics non sono reificati**: un oggetto di tipo generico perde, a runtime, i dettagli di rappresentazione che aveva a compile time. E' esattamente questa distinzione che spiega perche' non si possono creare array generici direttamente, e perche' `instanceof` su un tipo parametrizzato concreto sia vietato.

7.5.2 Perche' l'erasure: retrocompatibilita'

La scelta della type erasure ha mantenuto la **compatibilita'** con i programmi Java pre-generics (pre J2SE 5.0), permettendo l'introduzione dei generics senza richiedere modifiche alla JVM.

7.6 Confronto esplicito con i template C++ (da Wikipedia)

	C++ template	**Java generics**
Codice compilato	una versione **per ogni** istanziazione di tipo	**una sola** versione compilata, indipendentemente dai parametri di tipo usati
Specializzazione	si', supportata (cap. 5)	**non** supportata
Informazione di tipo	disponibile a compile time *e* usata per generare codice specifico	disponibile solo a compile time, poi **cancellata**
Sintassi "diamond"	`Box<>` = default template argument	`Box<>` = inferenza di tipo (Java 7+)

Fonte: Wikipedia, Generics in Java.

7.7 Sviluppi futuri: Project Valhalla

Il progetto **Valhalla** studia possibili miglioramenti futuri ai generics Java, tra cui: **generic specialization** (permettere `List<int>` invece di richiedere `List<Integer>`) e **reified generics** (rendere i tipi attuali disponibili a runtime). Questi miglioramenti dovrebbero convivere con la retrocompatibilita'.
Fonte: Wikipedia, Generics in Java.

Chapter 8

Genericita' in Python

8.1 Il punto di vista di Wikipedia: genericita' "gratuita"

La voce Wikipedia *Generic programming* osserva che nei linguaggi a **tipizzazione dinamica** il passaggio di valori alle funzioni e l'assegnazione di valori sono **type-indifferent**: la genericita' esiste gia' per costruzione. Questo di solito **non viene nemmeno chiamato "genericita"'**, perche' e' una conseguenza diretta del sistema di tipi dinamico, non un meccanismo dedicato come i template o i generics.
Fonte: Wikipedia, Generic programming.
Questo e' esattamente il punto della sezione "Container Classes Case Study" delle slide (capitolo 1), dove si diceva che in un linguaggio a tipizzazione dinamica le abstraction omogenee ed eterogenee "si costruiscono con relativa facilita'": le slide parlano di C++ e Java, ma la stessa osservazione vale per Python.

8.2 Duck typing

Definition 8.2.1: Duck typing (duck test)

"Se cammina come un'anatra e starnazza come un'anatra, allora deve essere un'anatra." Un oggetto e' considerato di un certo tipo se possiede **tutti i metodi e le proprieta'** richiesti da quel tipo, indipendentemente da qualunque dichiarazione formale di quel tipo.

Fonte: Wikipedia, Duck Typing.
La differenza rispetto alla **tipizzazione nominativa** (nominal typing): quest'ultima verifica il tipo tramite **dichiarazione esplicita** (es. `class MyBag(Sized)` in Python, o `implements`/`extends` in Java); il duck typing valuta il tipo in base ai **comportamenti effettivamente disponibili**. Python e' l'esempio principale di linguaggio dove il duck typing e' nativamente integrato (insieme, in misura diversa, a Ruby e JavaScript).
Fonte: Wikipedia, Duck Typing.

8.3 Il collegamento teorico con i template: lo stesso "duck test", due momenti diversi

Questo e' il punto di sintesi piu' importante di tutto il documento, e viene letteralmente dalla voce Wikipedia sul duck typing:

Proposition 8.3.1 Template e duck typing applicano lo stesso test, in momenti diversi

Le funzioni template (C++) applicano il *duck test* in un contesto di tipizzazione **statica**, mantenendo vantaggi e svantaggi di entrambi gli approcci. Il duck typing e' **piu' flessibile**, perche' richiede solo l'implementazione dei metodi *effettivamente invocati a runtime*; i template richiedono invece che siano valide *tutte* le implementazioni non provabilmente irraggiungibili a compile time.

Fonte: Wikipedia, Duck Typing.

Note:

Questa e' la stessa idea, vista da un'altra angolazione, del capitolo 2: "non c'e' type checking completo" alla definizione di un template, perche' il controllo e' rimandato all'istanziazione — che e' l'equivalente, a compile time, del momento in cui in Python l'`AttributeError` emergerebbe a runtime se un metodo mancasse davvero. Template e duck typing risolvono lo stesso problema ("non voglio impegnarmi su un'interfaccia esplicita") a due estremi opposti dell'asse compile-time/runtime.

8.4 Structural typing

Wikipedia distingue ulteriormente: il **structural typing** e' un sistema di tipizzazione *statico* che determina la compatibilita' in base alla **struttura** del tipo; il duck typing e' invece *dinamico* e valuta solo la parte di struttura effettivamente usata a runtime. Linguaggi come TypeScript, Elm e **Python** supportano il structural typing a vari livelli.

Fonte: Wikipedia, Duck Typing.

Nota: approfondimento non tratto da slide/Wikipedia, per completare il quadro.

In Python il modulo `typing` realizza concretamente questa idea con `Protocol`: una classe puo' "soddisfare" un protocollo avendo i metodi giusti, senza dichiarare mai un'ereditarieta' esplicita — e' duck typing formalizzato staticamente, verificabile da uno strumento esterno come `mypy`:

```python
from typing import Protocol, TypeVar, Generic

class Sized(Protocol):
    def __len__(self) -> int: ...

def describe(x: Sized) -> str:
    return f"size {len(x)}"

class MyBag:
    def __len__(self): return 42

describe(MyBag())   # OK per mypy: MyBag ha __len__, non serve ereditare da Sized

T = TypeVar("T")
class Box(Generic[T]):
    def __init__(self, item: T) -> None:
        self.item = item
```

Wrong Concept 8.1: I type hint di Python (`Generic[T]`, `TypeVar`) funzionano come i generics Java

No: a runtime, di default, Python **non controlla affatto** i type hint. `Box[int]("ciao")` costruisce felicemente un `Box` con dentro una stringa, senza alcun errore: i type hint finiscono solo nel dizionario `__annotations__` della funzione/classe, pura metadata. L'unico modo di intercettare una violazione e' eseguire uno strumento di analisi statica **offline** (`mypy`) prima di lanciare il programma — esattamente l'opposto del controllo obbligatorio e automatico che accompagna i generics Java o i template C++.

Chapter 9

Genericita' in C

9.1 "Function overloading, non true generic programming"

La voce Wikipedia *Generic programming* e' molto netta sul C: lo standard C11 offre le cosiddette **type-generic expression** tramite la keyword `_Generic`, che fornisce una forma di **overloading di funzione**, ma **non** vera programmazione generica.
Fonte: Wikipedia, Generic programming.
Questo conferma, con l'autorita' di una fonte esterna al corso, esattamente il limite discusso a proposito di `_Generic`: la selezione avviene a compile time fra un insieme **fisso e scritto a mano** di funzioni gia' esistenti (una per tipo elencato nella macro), non generazione automatica di codice per un tipo arbitrario futuro come farebbe un template.

9.2 L'esempio GNU C: macro + `typeof` + statement expression

Wikipedia mostra un esempio interessante di macro "generica" che risolve un problema classico:

```
#define max(a,b) \
       ({ typeof (a) _a = (a); \
          typeof (b) _b = (b); \
          _a > _b ? _a : _b; })
```

Fonte: Wikipedia, Generic programming.
Nota: approfondimento non tratto da slide/Wikipedia, per completare il quadro.
Questa macro e' istruttiva soprattutto per **cosa evita**: la classica macro `#define MAX(a,b) ((a)>(b)?(a):(b))` valuta i suoi argomenti **piu' volte** per sostituzione testuale (se `a` e' un'espressione con effetti collaterali come `i++`, viene eseguita due volte). Qui invece `_a` e `_b` sono variabili temporanee, create una sola volta dentro uno *statement expression* `({ ... })` — un'estensione GNU, non C standard — e `a`/`b` vengono quindi valutati esattamente **una volta** ciascuno. `typeof` (altra estensione GNU) deduce il tipo di `_a`/`_b` dall'espressione stessa, dando un minimo di "genericita' sui tipi" pur restando pura espansione testuale del preprocessore — senza alcun controllo di tipo reale al momento della scrittura della macro.

9.3 Il quadro completo delle tecniche C (slide + Wikipedia)

Il capitolo sulle slide C++ (sezione 1, "Pre OO Solutions for Container Classes") aveva gia' indicato, per i linguaggi a tipizzazione statica pre-OO come il C, le **variant record**/**union** come tecnica per container eterogenei, con il costo di un numero finito di alternative e di codice di test/cast "sporco". A quella tecnica si affiancano, storicamente:

- **macro del preprocessore** (sostituzione testuale, zero controllo di tipo, rischio di valutazione multipla come visto sopra);
- `void*` (puntatore generico, richiede cast manuali a ogni estrazione, zero type-safety — e' la tecnica usata da funzioni della libreria standard come `qsort`);

- `_Generic` (C11): dispatch a compile time su un insieme chiuso di tipi/funzioni scritte a mano.

Wrong Concept 9.1: `_Generic` e' l'equivalente C dei template

Secondo la stessa Wikipedia, no: `_Generic` da' overloading su un insieme **chiuso e predefinito** di funzioni, mentre un template (o un generic Java) si applica a **qualunque** tipo che soddisfi i requisiti richiesti, **compresi quelli che non esistevano ancora** quando il template e' stato scritto. Aggiungere un nuovo tipo a `_Generic` richiede di modificare a mano la macro; aggiungere un nuovo tipo a un template richiede solo che quel tipo supporti le operazioni usate nel corpo del template.

Chapter 10

Templates in Depth (dal libro di Eckel)

Il docente ha indicato come approfondimento il libro *Thinking in C++* di Bruce Eckel, liberamente accessibile. Il Vol. 2 dedica ai template l'intero **capitolo 5, "Templates in Depth"**, che presenta i template come un vero e proprio *sotto-linguaggio* dentro il C++: "i template fanno molto di piu' che creare container generici".
Fonte: Thinking in C++ *(Eckel), Vol. 2, cap. 5.*

10.1 Parametri di template: le quattro forme

Definition 10.1.1: Non-type parameter

Un template puo' essere parametrizzato non solo su un *tipo* ma anche su un **valore costante noto a compile time**: `template<class T, int sz>`. Il valore diventa **parte del tipo**.

Example 10.1.1 (Non-type parameter, verificato)

```
template <class T, int sz>
class Array {
    T data[sz];                  // la dimensione fa parte del TIPO
public:
    T& operator[](int i) { return data[i]; }
    int size() const { return sz; }
};

Array<int, 5>   a5;
Array<int, 100> a100;
```

Output verificato: `sizeof(Array<int,5>) = 20`, `sizeof(Array<int,100>) = 400` — sono **tipi diversi**. Un parametro non-type deve essere una **costante di compilazione**: `int n; cin » n; Array<int,n> a;` e' impossibile. E' esattamente il meccanismo di `std::array<T,N>`.

Fonte: Thinking in C++ *(Eckel), Vol. 2, cap. 5, "Non-type parameters".*
Verificato in `prove/T04_libro_nontype_typename.cpp` *(compilato ed eseguito).*

Definition 10.1.2: Default template argument

Come le funzioni, i template ammettono valori di default: `template<class T = int, int sz = 10>`, cosi' che `ArrayConDefault<>` sia gia' utilizzabile.

Definition 10.1.3: Template template parameter

Un parametro che e' esso stesso un *template* (non un tipo concreto), permettendo un'astrazione di livello superiore.

Proposition 10.1.1 Sono proprio queste le feature che mancano ai generics

Default template argument, non-type argument e template template argument sono tre delle cinque funzionalita' che Wikipedia elenca come **assenti** nei generics di Java e .NET (capitolo 11).

10.2 La keyword `typename` e i nomi dipendenti

Wrong Concept 10.1: `typename` e `class` sono sempre intercambiabili

Sono equivalenti *solo* nella lista dei parametri di template. `typename` ha un **secondo uso, non opzionale**: disambiguare i **nomi dipendenti** dentro il corpo di un template.

Fonte: Thinking in C++ *(Eckel), Vol. 2, cap. 5, "The typename keyword".*

Example 10.2.1 (Perche' serve `typename`)

```
template <typename Container>
void stampaPrimo(const Container& c) {
    typename Container::const_iterator it = c.begin();
    //  ^^^^^^^^ senza questo, errore oscuro
    std::cout << *it;
}
```

Senza `typename`, il compilatore assume di default che `Container::const_iterator` **non sia un tipo** (potrebbe essere un dato statico), e interpreta la riga come un'*espressione* (una moltiplicazione!). L'errore che ne risulta e' notoriamente incomprensibile.

Verificato in `prove/T04_libro_nontype_typename.cpp` (compilato ed eseguito).

10.3 Overloading e ordinamento parziale

Piu' function template possono coesistere: il compilatore sceglie il **match piu' specializzato**, secondo le regole di *partial ordering*. Il libro segnala anche una complicazione pratica: prendere l'**indirizzo** di una function template richiede un'istanziazione esplicita, perche' il compilatore non ha argomenti da cui dedurre i tipi.
Fonte: Thinking in C++ *(Eckel), Vol. 2, cap. 5, "Function templates".*

10.4 Gli idiomi: traits, policies, CRTP

Il capitolo dedica una sezione ai *template programming idioms*, cioe' agli usi dei template che vanno oltre il container generico.

Definition 10.4.1: Traits

Informazioni sul *tipo* codificate dentro classi template, che permettono **decisioni a compile time**: proprieta' numeriche, categorie di iteratori, codifiche di caratteri. Consentono ad un algoritmo di *adattarsi* al proprio parametro di template.

Definition 10.4.2: Policies

Parametri di template che **controllano il comportamento** di un algoritmo, permettendo un design flessibile **senza overhead a runtime** (l'esempio classico sono gli *allocator* dei container della STL).

Definition 10.4.3: CRTP — Curiously Recurring Template Pattern

Una classe **eredita da un template parametrizzato su se stessa**: `struct Der : Base<Der>`. Permette **polimorfismo statico senza funzioni virtuali** — si scambia il dispatch dinamico per un'astrazione a costo zero.

Fonte: Thinking in C++ *(Eckel), Vol. 2, cap. 5, "Template programming idioms".*

Example 10.4.1 (CRTP: polimorfismo senza vtable, verificato)

```
template <typename Derived>
struct Base {
    void interface() {
        static_cast<Derived*>(this)->implementation();  // dispatch STATICO
    }
};
struct Der1 : Base<Der1> {
    void implementation() { std::cout << "Der1\n"; }
};
```

Output verificato: `sizeof(Der1) = 1` byte — **nessun vptr**. Con una funzione virtuale sarebbero stati 8 byte (il vptr). Tutto il dispatch e' risolto a compile time e inlineabile.

Verificato in `prove/T05_libro_crtp_traits_metaprog.cpp` (compilato ed eseguito).

Note:

Collegamento con l'altro fascicolo: il CRTP e' l'esatto equivalente C++ dell'idioma Java `class Enum<E extends Enum<E>>` (*F-bounded polymorphism*). Stessa idea — una base generica che "sa" quale sia il tipo concreto che la deriva — realizzata con due strategie di genericita' opposte: monomorfizzazione (nessun residuo a runtime) contro erasure.

10.5 Metaprogramming ed expression templates

Oltre al fattoriale a compile time (capitolo 6), il libro introduce le **expression templates**: una tecnica sofisticata che *cattura le operazioni* dentro strutture template prima di valutarle, permettendo ottimizzazioni — per esempio, nell'aritmetica su vettori, **evitare la creazione di temporanei** intermedi. E' la base di librerie di calcolo numerico ad alte prestazioni come Eigen e Blitz++.

Fonte: Thinking in C++ *(Eckel), Vol. 2, cap. 5, "Template metaprogramming".*

10.6 I due modelli di compilazione

Definition 10.6.1: Inclusion model

Le definizioni dei template stanno negli **header**. Costringe a ricompilare tutto quando un template cambia, ma rende immediata la compilazione separata. E' il modello universalmente usato, ed e' il motivo della regola vista nel capitolo 2.

Definition 10.6.2: Separation model (`export`)

La keyword `export` avrebbe dovuto permettere di mettere le implementazioni dei template in file sorgente separati, risolvendo i problemi di efficienza di compilazione nei progetti grandi. Il libro la descrive come "raramente implementata" — ed e' infatti stata **rimossa in C++11** (capitolo 6).

Fonte: Thinking in C++ *(Eckel), Vol. 2, cap. 5, "Compilation models".*

Note:

Il libro avverte inoltre che i **nomi dentro i template** seguono regole di lookup complesse: i nomi dipendenti sono risolti in modo diverso, e le dichiarazioni `friend` dentro un template richiedono attenzione per mantenere le relazioni di accesso volute.

Chapter 11

Sintesi trasversale

Wikipedia sottolinea un punto su cui le slide non si spingono, ma che e' esattamente il tipo di domanda che il docente pone all'esame: template C++, generics Java e generics .NET sono spesso considerati simili, ma **i generics imitano solo il comportamento di base dei template C++**.
Fonte: Wikipedia, Template (C++).

Definition 11.0.1: Cosa manca ai generics rispetto ai template C++

Funzionalita' avanzate dei template C++ che i generics (Java, .NET) non offrono:

- **explicit specialization** (capitolo 5);
- **partial specialization** (solo per le classi);
- **default template argument**;
- **non-type template argument** (es. `Array<char, 6>`);
- **template template argument**.

Queste funzionalita' avanzate sono cio' che rende possibili librerie sofisticate di template metaprogramming come Boost, STLSoft e la STL stessa.

Note:

Storicamente, il concetto di template non era presente in linguaggi come Java e C# 1.0. Java ha poi adottato i **generics** (che imitano il comportamento di base dei template, ma tecnicamente sono un meccanismo diverso — *erasure* invece di generazione di codice); anche C# ha aggiunto i generics, in .NET 2.0. I generics di Ada, al contrario, **precedono** i template C++.

Fonte: Wikipedia, Template (C++).

11.1 Tabella comparativa a quattro vie

	C	**C++**	**Java**	**Python**
Meccanismo	nessuno strutturato: macro, `void*`, `_Generic` (insieme chiuso)	template: generazione di codice a compile time	generics: type erasure a compile time	nessuno: tipizzazione dinamica, duck typing
Quando si applica il duck test	mai in modo generale	a compile time, su chi verrebbe istanziato	a compile time, entro i bound dichiarati	a runtime, solo sui metodi davvero invocati
Codice a runtime	identico a quanto scritto a mano	una copia per tipo istanziato (monomorfizzazione)	una sola copia condivisa (erasure)	un solo oggetto codice, nessuna specializzazione
Type-safety	nessuna (cast manuali)	completa, ma solo dopo l'istanziazione	completa a compile time, persa a runtime	nessuna di default; opzionale e offline con mypy

11.2 Il filo che lega tutto

Il punto piu' importante da portare all'esame, che nessuna delle fonti isolate dice esplicitamente ma che emerge mettendole a confronto: template C++, generics Java e duck typing Python sono **tre risposte allo stesso identico problema** (evitare di duplicare codice per tipi diversi, capitolo 1), che differiscono solo per **quando** verificano che il tipo "vada bene" (compile time con generazione di codice, compile time con erasure, runtime senza generazione), non per **cosa** verificano (che il tipo supporti le operazioni richieste — il "duck test", capitolo 8). C, privo di entrambi i meccanismi, e' li' a ricordare cosa succede quando quel controllo, a qualunque livello, semplicemente **non c'e'**.

Chapter 12

L'esercizio proposto dal corso

Le slide concludono con un esercizio proposto, che chiede di completare l'implementazione di una `List<T>` e di uno `Stack<T>` generici.

Example 12.0.1 (Scheletro di `List<T>` da completare)

```
template <class T> class List {
public:
    void add(T);
    T firstElement;
    List<T> * nextElements;
};

List<int>    A;
List<double> B;

template <class T> int length(List<T> & X) {
    if (X==0) return 0;
    return 1 + length(X.nextElements);
}
```

Fonte: slide TemplatesInCpp, *pag. 39.*

Example 12.0.2 (Scheletro di `Stack<T>` da completare)

```
template <class T> class Stack {
public:
    void Push(T);
    T Pop();
};
template <class T> void Stack<T>::Push(T arg) {
    //...
};
template <class T> T Stack<T>::Pop() {
    //...
};

Stack<int>    s1;
Stack<double> s2;
s1.Push(1);
s1.Push(1.0);
```

Fonte: slide TemplatesInCpp, *pag. 40.*

Note:

Da notare: la sintassi `template <class T> void Stack<T>::Push(T arg) { ... }` per definire fuori dalla classe un metodo di una class template va sempre ripetuta con la keyword `template <class T>` anche nella definizione — esattamente come visto nella tecnicalita' declaration/definition/instantiation del capitolo 2: la *definizione* di `Push` e `Pop` e' essa stessa un template, non ancora una funzione concreta.

Chapter 13

Riepilogo

Concetto	Punto chiave
Motivazione	evitare la duplicazione di codice per container che differiscono solo per il tipo degli elementi (cap. 1)
Non fully generic	nessun type checking completo alla definizione; tutto e' rimandato all'istanziazione, per poter generare solo il codice necessario (cap. 2)
Function template	il compilatore *deduce* i parametri dagli argomenti della chiamata; alcuni errori (es. `operator*` mancante) emergono solo all'istanziazione (cap. 3)
Class template	nessuna deduzione: i parametri vanno sempre specificati; ogni istanziazione e' una classe indipendente e scorrelata dalle altre (cap. 4)
Specialization	si puo' sovrascrivere il codice generato per un tipo specifico (es. `char*` con `strcmp`); parziale solo per le classi (cap. 5)
Oltre le slide	variable template, non-type/template-template parameter, variadic template, alias, concepts, metaprogramming (cap. 6)
Java	generics con *type erasure*: una sola copia a runtime, wildcard e PECS, invarianza contro la covarianza degli array (cap. 7)
Python	nessun meccanismo dedicato: genericita' "gratuita" della tipizzazione dinamica, duck typing, `Protocol` come duck typing formalizzato (cap. 8)
C	nessuna vera genericita': macro (rischio di doppia valutazione), `void*`, `_Generic` (overloading su un insieme chiuso, non generazione di codice) (cap. 9)
Vs. generics	i generics (Java/.NET) imitano solo il comportamento base dei template: mancano specializzazione, default argument, non-type/template-template argument (cap. 11)
Il filo che lega tutto	template, generics ed erasure, e duck typing sono tre risposte allo stesso problema, che differiscono solo per *quando* verificano il "duck test" (cap. 11)

Fonti

- Slide del corso: *Templates in C++* (`20250922.TemplatesInCpp.pdf`), 40 pagine — fonte primaria per i capitoli 1–5 e 12.
- Wikipedia, *Template (C++)*, https://en.wikipedia.org/wiki/Template_(C++) — fonte per gli approfondimenti storici, le funzionalita' moderne del capitolo 6 e l'apertura del confronto con i generics del capitolo 11.
- Wikipedia, *Generics in Java*, https://en.wikipedia.org/wiki/Generics_in_Java — fonte primaria del capitolo 7.
- Wikipedia, *Generic programming*, https://en.wikipedia.org/wiki/Generic_programming — fonte per la genericita' "gratuita" di Python (cap. 8) e per il C (cap. 9).

- Wikipedia, *Duck typing*, https://en.wikipedia.org/wiki/Duck_typing — fonte primaria del capitolo 8, in particolare del collegamento teorico template/duck typing.
- Bruce Eckel, *Thinking in C++*, Vol. 2, cap. 5 (*Templates in Depth*), accessibile liberamente: https://web.mit.edu/merolish/ticpp/TicV2.html#_Toc53985706 — fonte primaria del capitolo 10.
- Cartella `prove/` (file `T01`–`T08`, `es_T01`–`es_T02`): ogni esempio citato con "Verificato in ..." e' stato **compilato ed eseguito** su questa macchina, non solo descritto.